USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

Raising the Flag: Detecting Missing Permission Controls in Mini-Program APIs

Zhiao Wei, *University of Waterloo;* Chao Wang, *The Ohio State University;* Haseeb-Ur-Rehman Faheem, *University of Waterloo;* Luyi Xing, *University of Illinois Urbana-Champaign;* Yousra Aafer, *University of Waterloo;* Zhiqiang Lin, *The Ohio State University*

https://www.usenix.org/conference/usenixsecurity26/presentation/wei-zhiao

This paper is included in the Proceedings of the 35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the 35th USENIX Security Symposium is sponsored by

جامعة الملك عبدالله للعلوم والتقنية

King Abdullah University of Science and Technology

Raising the Flag: Detecting Missing Permission Controls in Mini-Program APIs

Zhiao Wei*†, Chao Wang*‡, Haseeb-Ur-Rehman Faheem†, Luyi Xing§, Yousra Aafer†, Zhiqiang Lin‡
† *University of Waterloo*
‡ *The Ohio State University*
§ *University of Illinois Urbana-Champaign*

Abstract

Mini-programs embedded within super-apps like WeChat have surged in popularity due to their flexibility and convenience, offering rich functionalities through underlying APIs. However, this tight integration introduces serious security risks: mini-programs often inherit the mobile operating system's permissions granted to their host super-app, potentially bypassing critical security checks. In this paper, we address the urgent need for more fine-grained access control tailored to mini-programs. We propose PERMSCOPE, a systematic framework to detect and analyze missing scope checks in mini-program APIs, revealing instances where Android permissions are implicitly inherited and exercised in mini-program APIs, i.e., cases where "ask and check" primitives are absent. Our empirical analysis of four major super-apps reveals that 183 (8.85%) APIs are not properly protected at the mini-program API layer. We discuss the challenges underlying this issue and advocate for super-app developers to enforce fine-grained access control mechanisms at the API boundary, thereby strengthening the trustworthiness of the mobile super-app ecosystem.

1 Introduction

Mobile operating systems such as Android have improved user privacy and security through robust permission controls [26, 27, 35]. These controls ensure that apps must request explicit permissions to access sensitive user data (e.g., GPS coordinates) or system features (e.g., camera, microphone) before use. This model empowers users with informed consent and enforces accountability on app developers [25].

However, the rise of super-apps like WeChat, which consolidate multiple services via embedded "mini-programs," challenges this model. When users install a super-app, they often grant it a wide range of Android permissions (e.g., camera, location) to support the diverse functionalities offered by the app. These permissions are then inherited by all embedded mini-programs operating within the host app's privileged execution environment. From Android's perspective, actions performed by the super-app and mini-programs are indistinguishable.

While permission inheritance facilitates seamless user experiences, it introduces critical inconsistencies in access control. Official super-app documentation shows that *scopes*— super-app specific access control mechanism — are defined for only a limited set of permission-protected resources, including location, camera, and Bluetooth [7]. Many others, including network, NFC, or vibration, lack such scopes. This selective enforcement indicates that access control checks in super-apps are ad-hoc and insufficient, exposing users to unauthorized resource access.

The root of this problem is the absence of fine-grained scope enforcement at the API boundary of the mini-program API layer. Android's permission system cannot distinguish between host-initiated and mini-program-initiated operations, granting mini-programs implicit access to sensitive resources without user awareness. This undermines Android's foundational principle: granular control over access to user data and performed operations.

Therefore, this paper aims to *systematically detect and analyze missing scope checks in mini-program APIs*, focusing on instances where Android permissions are implicitly inherited and exercised. Specifically, we present PERMSCOPE, an *automated* dynamic analysis framework that identifies missing scope checks in mini-program APIs. PERMSCOPE tackles two core challenges: (i) generating valid test cases for each mini-program API, and (ii) tracing API execution flows across processes.

For challenge (i), we design a runtime testing environment guided by reverse-engineered API semantics and LLM-based parameter generation, enabling high-coverage invocation of mini-program APIs. For challenge (ii), we instrument super-apps to observe how mini-program API calls propagate through asynchronous handlers (e.g., `Handler.post(Runnable)`) and reach protected Android re-

*These authors contributed equally to this work.

sources. By linking these call chains, we detect APIs that access Android permission-protected resources.

We have evaluated PERMSCOPE across four major super-apps: WeChat [20], Baidu [12], Alipay [10], and QQ [18]. Our evaluation shows that a noticeable fraction (8.85%) of mini-program APIs exhibits what we call "access control gaps", stemming from three root causes: (1) *Partial Scope Enforcement*, (2) *No Scope Enforcement* (defined scopes but unchecked), and (3) *No Scope Definition* (entirely unguarded). Specifically, we identify 183 such APIs across 2,067 analyzed APIs, with WeChat exhibiting the highest number (96, 9.36%), followed by Alipay (43, 14.01%), Baidu (33, 6.51%), and QQ (11, 4.85%).

Contributions. Our key contributions are as follows:

- We highlight a systemic vulnerability in mini-program ecosystems, wherein mini-programs implicitly inherit super-app permissions through unprotected mini-program APIs.
- We develop PERMSCOPE, a framework for automatic detection of missing scope checks in mini-program APIs. Our method dynamically maps mini-program APIs to Android permission-protected resources through LLM-guided testing and cross-process instrumentation.
- We conduct a large-scale measurement study of 2,067 mini-program APIs across four major super-app platforms. Our results show that 183 APIs (8.85%) fail to enforce proper access control. We have disclosed all findings to the corresponding vendors. All vendors acknowledged our reports and remediated key vulnerabilities.
- We present detailed case studies illustrating how malicious mini-programs can exploit these gaps to infer user location, access private data, and control device hardware.

2 Background

2.1 Workflow of Mini-Program APIs

Mini-programs are a new class of mobile apps that leverage *web development technologies* (primarily JavaScript) while integrating native app capabilities to offer fast and seamless user experiences [5]. To support this hybrid runtime model, super-apps embed a custom execution environment on top of the system's web engine (e.g., WebView on Android [19] or JavaScriptCore on iOS [14]), with bridging components that translate JavaScript API calls into native operations. The mini-program runtime is structured into three layers:

- **Application Layer**: Developed in JavaScript by first- or third-party developers, this contains mini-program code and initiates all API calls.

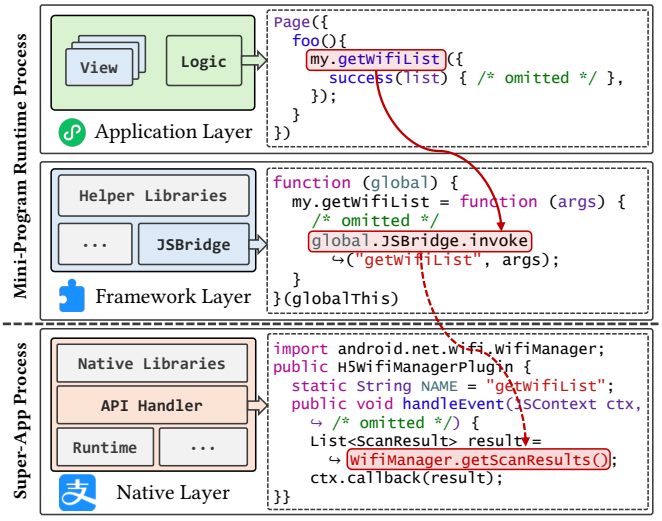

Figure 1: An example invocation flow of Alipay mini-program API `my.getWifiList`. The solid arrow means in-process invocation and the dashed means cross-process invocation.

- **JavaScript Framework Layer**: Written in JavaScript by super-app developers, this layer supports mini-program execution, sanitizes inputs, and translates API calls into a unified format.
- **Native Layer**: Written in C/C++ or Java, this layer handles API invocations, executes platform-level logic, and returns results to the upper layers.

Figure 1 illustrates an invocation of Alipay's mini-program API `my.getWifiList` [6]. At the Application Layer, a mini-program calls `my.getWifiList` to retrieve nearby Wi-Fi access points. This request is passed to the JavaScript Framework Layer, which wraps it into a unified format (`JSBridge.invoke("getWifiList", args)`) and forwards it to the Native Layer. There, the handler invokes `WifiManager.getScanResults` to retrieve the data, which is then returned to the mini-program through the stack.

As shown in Figure 1, the invocation chain across these layers creates a mapping between mini-program APIs and the underlying Android resources – an Android API in this example. When the Android resource does not require permissions, this arrangement is benign. However, when it requires permissions, a mismatch can arise because super-apps must implement access control equivalent to Android's permission enforcement.

Mini-program API invocations often involve execution paths spanning multiple processes. For example, a mini-program may execute in a dedicated runtime container while the API logic is handled in the app main process. Communication between these processes is typically facilitated through IPC mechanisms such as Android's Binder [3]. This

architectural separation complicates end-to-end analysis and tracing of permission-protected resource accesses triggered by mini-program APIs.

2.2 Permissions in Android and Super-Apps

Permissions in the Android operating system play a crucial role in regulating an app's access to device resources and user data. Specifically, Android's permission system provides users with granular control and transparency over what apps can access and perform on their devices. When a user installs an app, they are prompted to grant or deny requested permissions that belong to the *dangerous* protection level [4]. Requested permissions belonging to the *normal* protection level are granted automatically, but remain visible to users, providing transparency into the app's declared capabilities. For example, the presence of the requested `INTERNET` permission informs users that the app may potentially transmit data over the Internet.

In parallel, super-apps have implemented their own access control models to regulate mini-program behavior. These models use *scopes*, named access categories such as `scope.camera` or `scope.userLocation`, to define which mini-program APIs require user authorization. For instance, WeChat, Baidu, QQ, and Alipay each define their own scope systems and expose APIs (e.g., `wx.authorize`) that trigger runtime authorization dialogs when mini-programs request access to protected functionality.

Each scope typically corresponds to a resource class, such as device sensors, location data, or personal information. However, the granularity of protection differs across the platforms. For example, WeChat defines separate scopes for coarse, precise, and background location access, while Baidu uses a single location scope. This heterogeneity may lead to inconsistencies in how scopes are interpreted and enforced.

Beyond Permissions: the role of Intents. In addition to Android APIs, intents may provide mini-programs with indirect access to Android-protected functionality exposed by other apps. For example, the WeChat mini-program API `wx.sendSms` delegates SMS sending to the default SMS app – specifically, by launching the activity handling the intent `Intent("android.intent.action.SENDTO", Uri.parse("smsto:" + number))`. Similar delegation patterns are common across other mini-program APIs. For completeness, we therefore model intent-based communication with other app components as Android resource accesses and a potential source of access control gaps.

2.3 Comparison and Key Observations

Android permissions and super-app scopes differ significantly in both the protection granularity and the number of operations they regulate. As summarized in Table 1, the Android version considered in our study defines 180 permissions across two protection levels: 139 normal and 41 dangerous [11, 16] [1]. In contrast, the four super-app platforms we analyzed expose substantially fewer scopes: WeChat defines 14 scopes, Baidu defines 7, while both Alipay and QQ define 11. This disparity reveals a fundamental limitation: super-apps enforce only a narrow, coarse-grained subset of Android's broader permission space.

Platform		Protection Levels	Numbers of Permissions / Scopes	Example of Permission/ Scope
Android		normal	139	`ACCESS_NETWORK_STATE`
		dangerous	41	`BLUETOOTH_CONNECT`
Super-Apps	WeChat	normal	3	`invoice`
		dangerous	11	`userLocation`
	Baidu	normal	1	`invoiceTitle`
		dangerous	6	`userLocation`
	Alipay	normal	-	-
		dangerous	11	`location`
	QQ	normal	2	`invoice`
		dangerous	9	`userLocation`

Table 1: Comparison of Protection Levels and Counts across Android Permissions and Super-App Scopes. An example of each permission or scope is provided. - indicates that the super-app does not define the indicated protection level.

3 Overview

3.1 A Running Example

Figure 2 illustrates the issue of permission inheritance in super-apps using Alipay's `my.getWifiList` API.

❶ **Super-App Android Permission Configuration.** Super-apps typically request a broad set of permissions to support the diverse functionalities and services they provide. During their installation and initial use, the apps prompt users to approve the requested dangerous permissions (e.g., `CAMERA`, `LOCATION`). Once approved, these permissions are granted by Android to the super-apps [2].

As shown in Figure 2, Alipay requests the `LOCATION` permission to support location-based services, such as recommending nearby restaurants. Once granted, however, this permission is bound to the super-app itself rather than to individual mini-programs. Consequently, any mini-program launched within Alipay inherits `LOCATION` permission automatically, without triggering an additional consent dialog.

This form of implicit permission inheritance streamlines the user experience by limiting Android permission prompts to the initial use of the super-app. However, it also implies Android no longer mediates permission usage at the granularity of individual mini-programs. Resource accesses initiated by the host super-app and its mini-programs are indistinguishable.

[1] We exclude signature and system permissions as they cannot be obtained by (third-party) super apps.

[2] In contrast, normal permissions are granted automatically by Android.

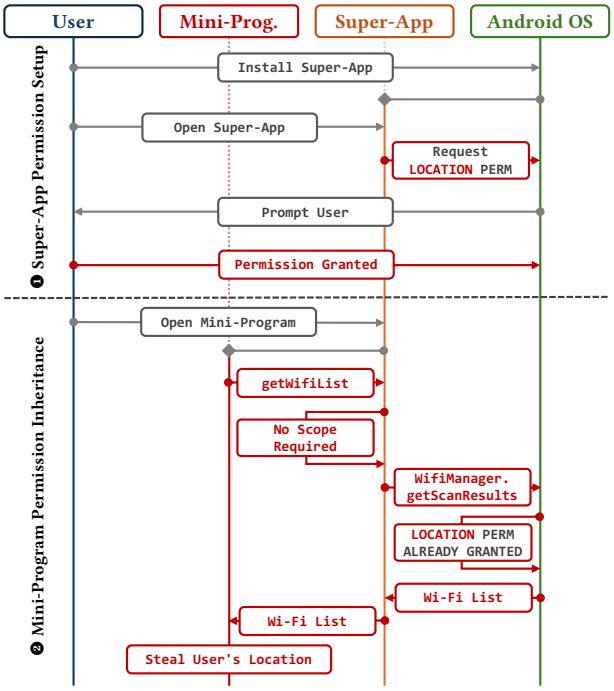

Figure 2: A running example of mini-program permission inheritance.

❷ Mini-Program Permission Inheritance. To address implicit permission inheritance, super-apps rely on their custom scopes to enforce access at the granularity of mini-programs. To maintain consistency, these scopes are intended to mirror the inherited Android permissions. However, as discussed in §2.3, scopes provide substantially less granularity and are far fewer in number than Android permissions, which can lead to access control gaps. Figure 1 illustrates one such example. Alipay defines a mini-program API `my.getWifiList`, which retrieves the nearby Wi-Fi list and metadata. Internally, the API invokes Android's `WifiManager.getScanResults`, which enforces `ACCESS_FINE_LOCATION` and `ACCESS_WIFI_STATE` permissions. However, we found that Alipay's API fails to properly enforce these requirements, as no dedicated scope exists to gate access to Wi-Fi information. This implies that once Alipay itself is granted the required Android permissions, any mini-program may invoke this unguarded API, enabling stealthy user tracking through Wi-Fi metadata [28]. We detail this vulnerability in §6.6.

3.2 Threat Model

We consider a threat model in which a malicious mini-program developer abuses the access control gaps described in §3.1 to illicitly access protected Android resources.

Attacker. The attacker is a mini-program developer who publishes an apparently benign mini-program through the standard submission channel of a super-app platform (e.g., the WeChat mini-program ecosystem). The mini-program passes the platform's review and becomes accessible to ordinary users through search, sharing, or in-app discovery.

Capabilities. The attacker can write a mini-program that invokes documented mini-program APIs exposed by the host super-app. The attacker has no special privileges on the victim device and operates entirely within the runtime environment that the super-app already provides to every mini-program.

Goal. The attacker aims to silently access resources protected by Android permissions that the host super-app has already obtained (at run-time or during installation time), without triggering the super-app's scope prompt or any Android runtime permission dialog.

Victim. The victim is an ordinary user who (1) has installed the super-app and granted it the Android permissions it requested, and (2) installed the attacker's mini-program.

Assumptions. We assume that (i) the super-app and the underlying Android system are trusted and not tampered with; (ii) the user has granted the super-app the Android-level permissions it requested, and (iii) the attacker's mini-program runs in the unmodified super-app runtime and shares the host's permission context.

3.3 Goals

A fundamental gap exists in super-app ecosystems due to the absence of publicly available, comprehensive mappings between mini-program APIs, the Android resources they access (e.g., APIs and Intent-based interfaces), and the Android permissions required to access those resources. This gap is further exacerbated by the mismatch between the granularity and number of mini-program scopes and Android permissions. As a result, it remains unclear whether the scopes enforced in mini-program APIs consistently align with the permissions required by the underlying Android resources.

To bridge this gap, this research aims to systematically detect such access control inconsistencies, thereby revealing vulnerabilities caused by the implicit permissions inherited from super-apps. We classify access control gaps into three categories of inheritance deficiencies, summarized in Table 2, and analyze their underlying causes:

Permission Inheritance Issue	Android Permission	Super-App Scope	Enforced by Mini-Program API?
Partial Scope Enforcement (PE)	**LOCATION**	`userLocation`	✓
	WIFI_STATE	N/A	✗
No Scope Enforcement (NE)	**BLUETOOTH**	`bluetooth`	✗
No Scope Definition (ND)	**FINGERPRINT**	N/A	✗

Table 2: Permission inheritance issues in super-apps.

- **Partial Scope Enforcement (PE).** The mini-program API enforces scope checks. However, these scopes cover only a subset of the Android permissions required to access the underlying resources.
  Example: Mini-program API `wx.getWifiList` requires `userLocation` scope, while the underlying Android API `WifiManager.getScanResults` requires both `ACCESS_FINE_LOCATION` and `ACCESS_WIFI_STATE` permissions, where the latter maps to `Wi-Fi` scope. Thus, the mini-program API exhibits partial scope enforcement, as it omits the required `Wi-Fi` scope.

- **No Scope Enforcement (NE).** The mini-program API entirely misses scope checks before accessing permission-protected Android resources. Although the scopes are defined by the super-app, they are never enforced by the mini-program API.

  Example: Mini-program API `my.getSystemInfo` performs no scope checks, while one of the underlying Android APIs requires `BLUETOOTH` permission

- **No Scope Definition (ND).** Similar to NE, the mini-program API accesses permission-protected Android resources without enforcing the corresponding scope check. However, unlike NE, the required scope is not defined by the super-app.

  Example: Mini-program API `wx.startSoterAuthentication` does not enforce any scopes. However, the underlying Android API `FingerprintManager.authenticate` requires the permission `USE_FINGERPRINT`. The super-app does not define a scope for this permission.

3.4 Analysis Scope

In this work, we analyze four popular super-apps: WeChat, Alipay, Baidu, and QQ. Unlike prior works that mainly perform cross-platform analysis and comparison [47], our work focuses on the Android framework to enable a deep analysis of how mini-program APIs reach permission-protected Android resources. In particular, this focus allows us to capture Android-specific subtleties, such as cross-process communication, and systematically identify and analyze inconsistencies between super-app scopes and Android's permission system.

3.5 Challenges and Insights

Mapping mini-program APIs to required Android permissions is a non-trivial task. In principle, static analysis could trace API calls to protected resources. However, super-apps exhibit complex runtime behaviors that are difficult to model statically, including dynamic parameters, user-driven interactions, runtime-dependent logic, and asynchronous IPC. Consequently, static techniques alone yield incomplete or inaccurate mappings, motivating the need for robust dynamic analysis.

Dynamic analysis overcomes these limitations by executing mini-program APIs within their runtime context, thus capturing their actual execution behavior. However, dynamically analyzing mini-program APIs entails two critical technical challenges:

(I) Generating Semantically Valid API Testcases. Dynamic analysis requires executing test cases that satisfy syntactic and semantic constraints of each API. However, generating automated test cases for mini-programs remains challenging. Existing solutions, such as APIDiff [47], often fail to trigger valid API execution at runtime. Empirically, we identify three primary causes of failure: (i) APIs that require user interactions (e.g., gestures); (ii) APIs that require initialization or depend on prerequisite operations before invocation; and (iii) APIs with strict parameter constraints that generic inputs fail to satisfy.

To address this, we employ an LLM-guided testing strategy that synthesizes semantically valid API invocations from official super-app documentation. We first generate initial candidate test cases using structured prompts derived from API specifications. We then execute these test cases, and analyze runtime failures to identify whether they stem from one of the following unmet execution conditions: user interaction, operation dependency or invalid parameter constraints. For execution-environment failures, we also use an LLM to generate dedicated mini-programs that incorporate the required interactions and dependent operations. For parameter-related failures, we iteratively refine prompts using error feedback. Section §4.2 details our dynamic testing framework.

(II) Handling Cross-Process API Invocations. Another inherent complexity of mini-program API testing arises from the asynchronous and cross-process nature of API invocations in super-app platforms. Mini-program APIs frequently involve interactions between mini-program runtime containers and the host app's main process through Android's Binder IPC mechanism. Traditional analysis methods, including static analysis and single-process runtime monitoring, struggle to capture execution flows spanning multiple processes, leading to incomplete mappings of mini-program APIs to Android permissions.

To address this challenge, we build PERMSCOPE on top of the Xposed framework [21], to instrument and reconstruct cross-process execution flows. Specifically, PERMSCOPE intercepts Binder IPC calls and `Handler.dispatchMessage` events to capture both cross-process and asynchronous execution flows. By correlating backtraces and thread identifiers across mini-program runtime containers and host processes, PERMSCOPE reconstructs end-to-end execution paths. This ensures accurate tracing of mini-program APIs to Android resources, enabling precise detection of access control gaps. §4.2 details this instrumentation.

4 Detailed Design

In this section, we present the design of PERMSCOPE. As shown in Figure 3, PERMSCOPE consists of three phases:

- **Mapping Android Resources to Android Permissions (§4.1).** This phase establishes the mapping between Android resources (framework APIs and Intent-based interfaces) and their required Android permissions.
- **Mapping Mini-Program APIs to Android Resources (§4.2).** In this phase, we generate semantically valid test cases from official API documentation using LLMs, while refining them to satisfy valid inputs and execution conditions. We then dynamically execute generated test cases while instrumenting super-apps to capture the invoked Android resources, thereby building the mapping between mini-program APIs and Android resources.
- **Mapping Mini-Program APIs to Android permissions (§4.3).** We derive the final mapping by combining the outputs of the previous phases. Specifically, we propagate the permissions associated with invoked Android resources back to the corresponding mini-program APIs.

4.1 Mapping Android Resources to Android Permissions

To build the mappings, we begin by identifying Android resources that can be accessed by super-apps and corresponding mini-program APIs.

Framework APIs. We statically analyze Android AOSP (version 14) to recover public APIs exposed through Android system services, including both officially documented SDK APIs and undocumented framework APIs. Specifically, we first extract system services registered in the `ServiceManager` registry and recover the APIs exposed through their corresponding client-facing service interfaces. To obtain the permission requirement for these APIs, we statically analyze their implementations using the prior path-sensitive approaches for constructing API-to-permission mappings [23, 24]. Although Android apps may directly invoke these recovered APIs, they more commonly interact with higher-level framework wrapper APIs that internally invoke them. Therefore, we further expand our API set by performing reachability analysis to identify such higher-level framework wrappers.

Intent-based interfaces. For Intent-based interfaces, we first use the official Android framework definitions of the `Intent` class to identify common framework-defined Intent actions that may be triggered by mini-program APIs [13]. We then approximate their permission requirements by extracting the associated *@RequiresPermission* annotations. This maps common SDK-exposed Intent-based resources to the Android permissions required to access them. While not exhaustive, this provides a conservative mapping for common permission-protected Intent-based resources.

4.2 Mapping Mini-Program APIs to Android Resources

This phase uncovers Android resources that are reachable from a given mini-program API. The phase consists of two steps: generating valid test cases and hooking the recovered set of Android APIs and Intent-launching mechanisms.

Generating Valid Test Cases. A critical prerequisite for dynamic analysis is generating valid test cases that successfully exercise mini-program APIs. We leverage LLMs to interpret official API documentation and synthesize test cases that satisfy API-specific syntactic and semantic constraints.

Documentation preprocessing. The four evaluated platforms (WeChat, Alipay, Baidu, and QQ) document their mini-program APIs using distinct website, specification structures and documentation formats. To provide a unified input representation for test-case generation, we develop a platform-specific crawler for each super-app that extracts and translates all documented APIs into a normalized JSON schema. Each API entry records the API name, signature (`definition`), textual description, documentation category, and parameter specifications. For each parameter, we extract its `type`, whether it is `required`, its `defaultValue`, textual `description`, and the minimum supported platform version (`minimumSupport`). We further classify APIs as `ASYNC` when their parameters include callback handlers such as `success`, `fail`, and `complete`. APIs without such callbacks are classified as `SYNC`. Overall, this preprocessing step produces 2,067 normalized API specifications, including 1,026 from WeChat, 507 from Baidu, 307 from Alipay, and 227 from QQ.

Prompt construction. We design a prompt template for each evaluated super-app. Each template consists of three parts. First, the prompt includes a task description that specifies the global JavaScript object for the target platform (e.g., `wx`, `my`, `swan`, or `qq`), and instructs the model to generate executable test cases for a target mini-program API. Second, the prompt includes several worked examples that describe the expected output format. Each generated test case consists of two components: a metadata comment followed by a single-line JavaScript invocation. Synchronous APIs are represented as direct function calls, whereas asynchronous APIs are wrapped in an immediately invoked function expression (IIFE) whose `success`, `fail`, and `complete` callbacks record execution results in `globalThis.resultSet`. Third, the prompt constrains parameter generation to documented API specifications. Specifically, the model is instructed to populate required parameters using the semantic information provided in the parameter `description` and API `definition`, reuse documented `defaultValue` whenever available, and fall back to generic placeholder values only when the documentation does not provide sufficient constraints. We release the complete prompt templates as part of our open-science artifact (§10).

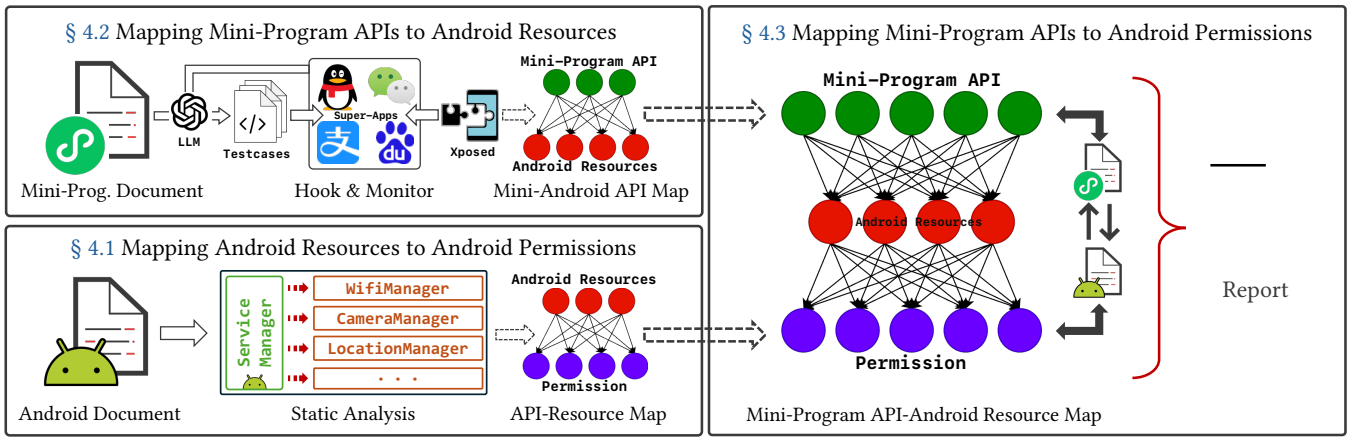

Figure 3: The analysis pipeline of PERMSCOPE.

As discussed earlier, the execution of some test cases resulted in two types of failures. First, some APIs require explicit user interaction, such as TAP gestures, to execute successfully. For example, `wx.addPhoneCalendar` fails with the message `"can only be invoked by user TAP gesture"` when executed without a tap. Second, certain APIs depend on the prior execution of other operations. For instance, `wx.makeBluetoothPair` requires `wx.openBluetoothAdapter` to initialize the Bluetooth adapter beforehand, whereas `wx.connectWifi` requires Wi-Fi to be enabled in advance.

To address these failures, we design a pipeline that combines structured API documentation (in JSON format) with runtime error feedback in LLM prompts (example prompts are provided in §10). For parameter-related failures, the LLM proposes alternative inputs that satisfy documented parameter constraints. For user interaction and dependency-related failures, we prompt the LLM to synthesize test mini-programs that include the required user interface elements (e.g., buttons for TAP gestures) and prerequisite operations (e.g., enabling Bluetooth before pairing). These generated mini-programs are then manually executed and validated.

Example. As shown in Figure 4, the initial test case for `wx.cropImage` used placeholder values such as `"src":"1"` and `"cropScale":"1"`, causing runtime errors. Using refined prompts grounded in the official API documentation, the pipeline subsequently generated semantically valid replacements (e.g., `"src":"wxfile://tmp.jpg"` and `"cropScale":"1:1"`), leading to a successful invocation.

Hooking and Monitoring Android Resource Invocation. With valid test cases in place, the next step is to determine which Android APIs and intents are triggered by each mini-program API. As discussed in §3.5, a key challenge is that many invocations span multiple processes. In WeChat, for example, a mini-program runs in `com.tencent.mm:mini` while

```
// Failing test case
(() => {
  wx.cropImage({
    "src": "1",
    "cropScale": "1",
    "success": function (e) { /* omitted */ },
    "fail": function (e) { /* omitted */ },
    "complete": function (e) { /* omitted */ },
  })
})()

// Successful test case: Correct replacements suggested by LLMs
(() => {
  wx.cropImage({
    "src": "wxfile://tmp.jpg",
    "cropScale": "1:1",
    "success": function (e) { /* omitted */ },
    "fail": function (e) { /* omitted */ },
    "complete": function (e) { /* omitted */ },
  })
})()
```

Figure 4: LLM-Guided Test Case Generation for `wx.cropImage`

API execution logic resides in `com.tencent.mm`, with tasks forwarded via Binder IPC.

To address this challenge, we use Xposed [21] to instrument super-apps and reconstruct mini-program API execution flows. Our approach proceeds in four steps:

i) **Identify and Hook Java Bridge Classes.** Using the JEB decompiler [15], we locate platform-specific Java packages that bridge calls from the JavaScript layer to the Android framework. For example, WeChat routes through `com.tencent.mm.plugin.appbrand.jsapi`, Baidu through `com.baidu.swan.apps.api`, Alipay through `com.alibaba.ariver.commonability`, and QQ through `com.tencent.qqmini`. By hooking methods within these packages, we intercept mini-program API invocations at their entry points. Table 3 summa-

Super-App	API Naming Convention	Mini-Program API Class Path	Main Hook Target	Android Invocation Method
WeChat	`wx.<MiniAPIName>`	`com.tencent.mm.plugin.appbrand.jsapi.*`	`MMHandler.post()`, `dispatchMessage()`	Multithreaded task dispatch
Baidu	`swan.<MiniAPIName>`	`com.baidu.swan.apps.api.*`	`Module class methods`, `SwanAppActivity lifecycle`	Module encapsulation or base class, Activity launch
Alipay	`my.<MiniAPIName>`	`com.alibaba.ariver.commonability.*`	`BridgeExtension`	Unified interface dispatch, component invocation
QQ	`qq.<MiniAPIName>`	`com.tencent.qqmini.*`	`performAction()`, `RequestEvent`	Lifecycle-driven, action-based dispatch

Table 3: Hooking strategies for super-apps.

rizes platform differences in naming conventions, bridge architectures, and hooking strategies.

ii) **Capture Cross-Process Dispatch.** We instrument message dispatching routines such as `Handler.post` and `dispatchMessage` to observe when tasks are created in the runtime process and executed in the main process.

iii) **Reconstruct Execution Flows.** For each intercepted call, we collect backtraces, thread identifiers, and method arguments from both the runtime and main processes. Using thread IDs and message objects as correlation hints, we correlate these traces to reconstruct unified cross-process execution flows linking mini-program APIs with their underlying Android resource invocation.

iv) **Log API Invocation Details.** All intercepted invocations are logged with metadata, including method signatures, argument values, return results, and timestamps.

This hooking and monitoring framework provides a reliable mapping of mini-program APIs to Android resources.

4.3 Mapping Mini-Program APIs to Android Permissions

With the mapping between mini-program APIs and their corresponding Android resources established in §4.2, the final step is to link the mini-program APIs to the Android permissions protecting the reachable resources.

This phase combines two mappings: (i) the Android Resource → Android Permission mapping (*R2P*) derived in §4.1, and (ii) the Mini-Program API → Android Resource mapping (*M2R*) derived in §4.2.

Formally, we define the Mini-program API → Android Permission mapping (*M2P*) as:

$$M2P(m) = \bigcup_{r \in M2R(m)} R2P(r), \quad \forall m \in M$$

This can be seen as the composition $M2P = R2P \circ M2R$ followed by set-union aggregation. In practice, we implement this as a *hash join* on the Android resource dimension, and use hash sets to guarantee uniqueness of permissions. Each mini-program API can be processed independently of the others, allowing the join to scale linearly. Algorithm 1 shows the algorithm, which performs a parallelizable join between *M2R* and *R2P* keyed on Android resource identifiers, with a total complexity of $O(|M| + |E|)$ where E is the total number of Mini-program-API-to-Resource edges observed.

Algorithm 1: Mapping Mini-Program APIs to Android Permissions via Join Composition

Input: M: Set of mini-program APIs
$R2P$: Hash map of Android resources → Permission sets
$M2R$: Hash map of mini-program APIs → List of Android Resources
Output: $M2P$: Hash map of mini-program APIs → Permission sets

```
for m ∈ M do in parallel
    permissions ← ∅;
    for r ∈ M2R[m] do
        if r ∈ R2P then
            permissions ← permissions ∪ R2P[r];
    if permissions ≠ ∅ then
        M2P[m] ← permissions;
return M2P
```

5 Implementation

We built a prototype of PERMSCOPE that consists of two main components: (1) A static analysis component built on top of WALA [9], responsible for analyzing Android Resources and extracting their permissions. (2) A dynamic analysis component based on Xposed, consisting of two main modules: *i*) an API Analysis Module that invokes mini-program APIs to observe triggered Android resources; and *ii*) a Message Queue and Thread Handling Module that analyzes multi-threaded dispatch mechanisms. It accomplishes this by intercepting specific methods to track and analyze objects within the message queue. For validation, we reverse-engineered each super-app using JEB decompiler [15].

To ensure that the testing process met the prerequisite conditions for API invocation, we granted all scopes in the super-app settings to the mini-programs during testing. For scenarios requiring user tap gestures or the prior invocation of

specific APIs, we generated dedicated mini-programs helpers for each case and conducted targeted manual testing. Each helper mini-program is typically under 50 lines.

6 Evaluation

In this section, we evaluate PERMSCOPE across four widely used super-app platforms: WeChat, Alipay, Baidu, and QQ. We organize the evaluation around the following research questions.

RQ1. To what extent can PERMSCOPE reliably generate valid test cases for mini-program APIs?

RQ2. What is the landscape of mappings between mini-program APIs and Android permission-protected resources across super-apps?

RQ3. How prevalent are access control gaps, and what types of permission inheritance deficiencies cause them?

RQ4. Which Android permissions lack corresponding scopes in super-apps?

RQ5. How consistently do super-apps enforce scopes for similar mini-program APIs?

6.1 Evaluation Results

Results of test case generation. Table 4 summarizes the results of our automated test-case generation and execution across four super-app platforms. In total, we generated and executed test cases for 2,067 mini-program APIs. To evaluate effectiveness, we compared PERMSCOPE against APIDIFF [47] for WeChat and a structure-aware random generator for other super-apps. The reported pass rates for PERMSCOPE correspond to the first execution round.

As shown, PERMSCOPE achieved higher pass rates (94.82–99.56% across the three LLMs) compared to APIDIFF's 80.02% and approximately 63% for the structure-aware random generator.

Robustness across different LLMs. To verify that our approach does not depend on a specific LLM, we run the test-case generation with multiple models. We ran PERMSCOPE with Claude Opus 4.7, DeepSeek-R1, and GPT-4o, using the same prompts and normalized API documentation, and compared the resulting pass rates (Table 4). All three models achieved high pass rates: Opus 4.7 reached 96.42–99.56% across the four platforms, DeepSeek-R1 reached 94.82–98.59%, and GPT-4o reached 96.21%–98.90%. The slightly lower pass rates for DeepSeek-R1 were primarily caused by a few structural generation errors in its one-round output, such as including unmatched delimiters in long callback bodies and method calls on uninitialized objects. Claude Opus 4.7 consistently generated structurally valid outputs for all APIs.

Failed test cases. Table 4 further reports a breakdown of the observed failed test cases: (1) APIs requiring UI interactions that are not automated and (2) missing dependent operations. WeChat exhibited the highest number of failures (20 cases), while QQ exhibited the lowest (1 case). To address these failures, we implemented dedicated mini-programs for each failed test case (38 mini-programs in total) and manually executed the tests.

Reliability and Scalability. To evaluate reliability, we report the number of refinement rounds required for successful test-case generation. All APIs converged within at most three refinement rounds. Across the four super-apps, this resulted in at most approximately $3 \times 2{,}067$ LLM-driven test-case generation attempts. The total monetary cost of LLM-based test-case generation was approximately 55 USD.

Answer to RQ1: PERMSCOPE reliably generates valid test cases for mini-program APIs, with all APIs converging within at most three refinement rounds.

Platform	# Test cases	Pass Rate					Failure Type	
		Random	APIDIFF	Opus 4.7	Deep Seek	GPT-4o	UI action	Dep Op
WeChat	1,026	-	80.02%	**98.05%**	**95.46%**	**97.93%**	9	11
Baidu	507	63.34%	-	**98.82%**	**97.92%**	**98.36%**	1	5
Alipay	307	59.17%	-	**96.42%**	**94.82%**	**96.21%**	1	10
QQ	227	66.96%	-	**99.56%**	**98.59%**	**98.90%**	1	0

Table 4: Results of automated test case generation.

6.2 Breakdown of Mini-Program API to Android Permission Mappings

We next analyze the PERMSCOPE-generated mappings between mini-program APIs and the underlying Android permissions. These mappings form the foundation for detecting permission inheritance deficiencies.

Table 5 reports the number of mini-program APIs that lead to an Android permission-protected resource (through either an Android API invocation or Intent-based interaction). Note that this mapping excludes mini-program APIs that only access non-protected Android resources. As shown, the ratios vary across the four super-apps, ranging from 9.5% in Baidu to 19.5% in Alipay.

Consistency Landscape. Using the recovered Mini-Program API-to-Permission mappings, we analyze whether the scopes enforced by super-apps properly reflect the underlying Android permissions. To this end, we first crawl the developer documentation of each super-app [10, 12, 17, 18] to obtain the scopes enforced by mini-program APIs. We then compare these scopes against the Android permissions required by the reachable Android APIs. Since super-app scopes are significantly fewer and coarser-grained than Android permissions,

Super-App	Android Resources			Access Control	
	API	Intent	Both	Consistent	Inconsistent
WeChat	118	11	1	32	96
Baidu	40	9	1	15	33
Alipay	52	10	2	17	43
QQ	15	7	0	11	11
Total	**225**	**37**	**4**	**75**	**183**

Table 5: Breakdown of mini-program APIs reaching Android permission-protected resources and their access control landscape.

performing a strict one-to-one matching would be both difficult and unfair. Hence, we perform a high-level semantic comparison by categorizing both scopes and Android permissions into broader protection categories based on the functionality they mediate access to, and comparing them at the category level rather than by exact literals. More details on this categorization are discussed in §6.3.

Our analysis shows that 75 of the mini-program APIs are consistent, meaning they enforce scopes aligned with the required Android permissions. However, the remaining cases exhibit inconsistencies, where mini-programs may silently access Android permission-protected resources without acquiring a corresponding scope.

Answer to RQ2: Across the four evaluated super-apps, PERMSCOPE recovered 258 mini-program APIs that reach an Android protected resource. Among them, only 75 (29.07%) enforce a scope aligning with the permission requirement.

Next, we provide a detailed breakdown of these access control gaps.

6.3 Breakdown of Access Control Gaps

Access Control Gaps per Super-app. Table 5 further breaks down the gaps by super-app. Overall, we identified 96 affected mini-program APIs in WeChat, 33 in Baidu, 43 in Alipay, and 11 in QQ that fail to enforce the appropriate scope checks. This indicates that access control gaps are prevalent across all four platforms.

Access Control Gaps per Protection Category. Table 6 reports a breakdown of the identified mini-program APIs across the four evaluated platforms. As shown, the identified access control gaps are widespread across nearly all evaluated protection categories. In particular, APIs associated with Network, Wi-Fi, Vibrate, and NFC protection exhibit consistently high gap rates, often reaching 100%. Sensitive protections such as Camera and Location also show substantial inconsistency, averaging 68% (location) and 48% (camera) missing scopes.

Scope Type	WeChat			Baidu			Alipay			QQ		
	M	T	%	M	T	%	M	T	%	M	T	%
Location	8	12	66.67	5	7	71.42	8	10	80.00	0	2	0.00
Location (Fuzzy)	17	18	94.44	-	-	-	13	13	100.00	-	-	-
Location (BG.)	0	1	0.00	-	-	-	-	-	-	-	-	-
Recorder	0	2	0.00	0	1	0.00	0	2	0.00	0	1	0.00
Camera	4	12	33.33	6	8	75.00	0	2	0.00	2	3	66.67
Bluetooth	6	8	75.00	2	2	100.00	3	6	50.00	2	2	100.00
Contacts	0	2	0.00	1	1	100.00	1	1	100.00	-	-	-
Calendar	0	2	0.00	0	2	0.00	-	-	-	-	-	-
Network	11	11	100.00	8	8	100.00	6	6	100.00	2	2	100.00
Wi-Fi	13	13	100.00	9	9	100.00	9	9	100.00	3	3	100.00
Fingerprint	3	3	100.00	-	-	-	-	-	-	-	-	-
Vibrate	2	2	100.00	2	2	100.00	3	3	100.00	2	2	100.00
Screen Lock	1	1	100.00	-	-	-	-	-	-	-	-	-
NFC	31	31	100.00	-	-	-	-	-	-	-	-	-
Total	**96**	**118**	**81.36**	**33**	**40**	**82.50**	**43**	**52**	**82.69**	**11**	**15**	**73.33**

Table 6: Access control gaps Mini-program APIs broken down by corresponding Android permission category. "M" denotes the number of Mini-program APIs that miss the scope checks; "T" denotes the number of Mini-program APIs that require the scope checks; "-" indicates that the super-app does not define APIs requiring the protection;

Severity by Android Protection Level. To better understand the security implications of the identified access control gaps, we report a breakdown of the 183 findings according to the protection level of their underlying Android permissions. The findings span both dangerous and normal protection levels (note that PERMSCOPE excludes system/signature protections as they cannot be acquired by super-apps). As shown in Table 7, overall, 81 of the 183 findings involve dangerous-level permissions, while the remaining 102 involve normal-level permissions.

We consider the instances involving dangerous permissions as higher severity because these permissions protect user-identifiable data or sensitive operations (e.g., camera or fingerprint-related functionality). In contrast, cases involving normal permissions are lower severity, but still security-relevant. Prior work [30, 42] has demonstrated that normal-level permissions can be systematically abused to leak user-identifying data and fingerprint mobile users [42]. The work [53] similarly shows that the normal-level *Bluetooth* permission can be abused.

Access Control Gaps per Inheritance Issue. Table 8 groups the identified gaps by permission inheritance issue type. We next discuss the results for each category in detail.

Protection Level	WeChat	Baidu	Alipay	QQ	Total
Dangerous	38	14	25	4	81
Normal	58	19	18	7	102
Total	96	33	43	11	183

Table 7: Severity distribution by Android protection level.

Super-App	PE	NE	ND	Total
WeChat	13	27	56	96
Baidu	10	10	13	33
Alipay	12	7	24	43
QQ	0	2	9	11
Total	**35**	**46**	**102**	**183**

Table 8: Access control gaps grouped by permission inheritance issue type: Partial Scope Enforcement (PE), No Scope Enforcement (NE), and No Scope Definition (ND).

Partial Scope Enforcement (PE). PERMSCOPE identified **35 instances** of PE issues. Recall that these cases correspond to mini-program APIs that only partially enforce the scopes required for accessing the underlying Android resources. WeChat exhibited the highest number of cases (13), followed by Alipay (12) and Baidu (10). No such cases were observed in QQ.

No Scope Enforcement (NE). This category corresponds to mini-program APIs that access underlying permission-protected Android resources without enforcing any corresponding scope, despite a matching scope being defined for the permission.

PERMSCOPE identified **46 such instances** on the four platforms: 27 on WeChat, 10 on Baidu, 7 on Alipay, and 2 on QQ. Examples include camera-related mini-program APIs such as `wx.scanCode` and `swan.chooseVideo`. While these APIs invoke Android APIs protected by the `CAMERA` permission, they do not enforce the *Camera* scope. Consequently, mini-programs can invoke these APIs without triggering any user prompt for camera access approval.

No Scope Definition (ND). Similar to NE, this category involves mini-program APIs that access permission-protected resources without scope enforcement; however, no corresponding scope is defined for the required permissions.

PERMSCOPE revealed **102 ND instances**. This issue is especially pronounced in WeChat, which accounted for 56 cases, more than half of all observed instances. Alipay, Baidu, and QQ exhibited 24, 13, and 9 cases, respectively. Mini-prpgram APIs in such cases cannot intrinsically enforce scope checks. Examples include `wx.vibrateShort` and `qq.vibrateLong`, which all miss Android's `VIBRATE` permission.

Answer to RQ3: Across the four super-apps, PERMSCOPE identified 183 access control gaps, affecting 8.85% of the analyzed mini-program APIs. The gaps span three types of permission inheritance mismatches: Partial Scope Enforcement (PE), No Scope Enforcement (NE), and No Scope Definition (ND), with ND being the most prevalent.

6.4 Permissions Without Scopes

To understand the root causes of ND cases, we analyze whether Android permission-protected resources have corresponding scope definitions in each super-app. Specifically, for each Android resource reachable from the four mini-program APIs, we map its required Android permission into higher-level functionality categories and examine whether the super-apps define a corresponding scope. Table 9 lists the categories. As shown, while some functionalities have a scope in all platforms (e.g., camera and microphone access), others lack corresponding scopes. For example, Network, Wi-Fi, Fingerprint, Vibrate, and NFC functionalities are not associated with any scope definition across all evaluated super apps. Other functionalities, such as Bluetooth, have a corresponding scope only in a subset of platforms. These missing scope definitions directly contribute to the prevalence of ND cases across the super-app ecosystem.

Answer to RQ4: Several Android permissions, including `ACCESS_NETWORK_STATE`, `ACCESS_WIFI_STATE`, `USE_FINGERPRINT`, `VIBRATE`, and `NFC`, lack corresponding scopes across all four super-apps. Others are inconsistently associated with scopes across the platforms.

6.5 Super-Apps Scopes Comparison

Building on our recovered Mini-Program API-to-Permission mappings, we next answer RQ5: *how consistently do different super-apps enforce scopes for semantically similar mini-program APIs?*. This comparison is important because super-app ecosystems expose many overlapping mini-program functionalities (e.g., location access, camera usage), yet each super-app independently designs and enforces its own scope system. Consequently, the same underlying Android permission may be properly protected in one super-app while remaining weakly protected or entirely unprotected in another.

To perform this analysis, we identify semantically equivalent mini-program APIs across the four super-apps and compare their corresponding scope enforcement against the required Android permissions.

As detailed in Table 10, we identified 38 semantically aligned mini-program APIs across the four super-apps and grouped them into six functionality categories, although not all super-apps define APIs for every category. These categories serve as normalized scopes for our comparison, avoiding reliance on platform-specific scope literals.

Our comparative analysis revealed discrepancies in scope enforcement across the four platforms, which clearly indicates that the current super-app ecosystem lacks a unified access control model. For instance, the mini-program API "`chooseImage`", which provides similar functionality across all four super-apps, is protected inconsistently. Specifically,

Functionality Category	Super-App Scopes								Android Permission
	WeChat		Baidu		Alipay		QQ		
Location	userLocation	✓	userLocation	✓	location	✓	location	✓	ACCESS_FINE_LOCATION
	userFuzzyLocation	✓	-	-	-	✗	-	-	ACCESS_COARSE_LOCATION
	userLocationBackground	✓	-	-	-	-	-	-	ACCESS_BACKGROUND_LOCATION
Recorder	record	✓	record	✓	audiorecord	✓	record	✓	RECORD_AUDIO
Camera	camera	✓	camera	✓	camera	✓	camera	✓	CAMERA
Bluetooth	bluetooth	✓	-	✗	bluetooth	✓	-	✗	BLUETOOTH_SCAN
Contacts	addPhoneContact	✓	-	✗	-	✗	-	-	WRITE_CONTACTS
Calendar	addPhoneCalendar	✓	calendar	✓	-	-	-	-	WRITE_CALENDAR
Network	-	✗	-	✗	-	✗	-	✗	ACCESS_NETWORK_STATE
Wi-Fi	-	✗	-	✗	-	✗	-	✗	ACCESS_WIFI_STATE
Fingerprint	-	✗	-	-	-	-	-	-	USE_FINGERPRINT
Vibrate	-	✗	-	✗	-	✗	-	✗	VIBRATE
Screen Lock	-	✗	-	-	-	-	-	-	WAKE_LOCK
NFC	-	✗	-	-	-	-	-	-	NFC
Address Info	address	✗	address	✓	aliaddress	✓	address	✓	N/A
Exercise Info	werun	✓	-	-	alipaysports	✓	qqrun	✓	N/A
User Info	userInfo	✓	-	-	-	-	userInfo	✓	N/A
Album	writePhotosAlbum	✓	writePhotosAlbum	✓	album	✓	writePhotosAlbum	✓	N/A
					writePhotosAlbum	✓			
Device Info	-	✗	-	✗	Device Info	✓	-	✗	N/A
Phone Number	-	-	-	-	phoneNumber	✓	-	-	N/A
Invoice	invoice/invoiceTitle	✗	invoiceTitle	✓	-	-	-	-	N/A

Table 9: correspondence of super-app scope definitions to Android permission across the four evaluated platforms; "✓" indicates the super-app introduces a mini-program API that requires the scope. "✗" indicates that the super-app introduces an API that requires the scope but does not define it. "-" under the the first column indicates that the super-app does not define the scope, while a "-" under the second column indicates that the super-app does not introduce APIs that require the scope.

PERMSCOPE-generated mapping shows that invoking "chooseImage" ultimately reaches a series of Android camera-related APIs "android.hardware.Camera.*", which require the CAMERA permission. However, only Alipay and QQ enforce the corresponding *Camera* scope, whereas WeChat and Baidu do not enforce it at all.

Answer to RQ5: Scope enforcement is inconsistently implemented for similar functionality across the four evaluated super-apps.

6.6 Case Studies

Location Tracking via getWifiList in Alipay. As shown in Figure 5, a malicious mini-program in Alipay can exploit missing location scope to infer user location without consent. The mini-program API my.getWifiList directly invokes WifiManager.getScanResults, which requires ACCESS_FINE_LOCATION. However, Alipay does not prompt the user for the location scope. Once invoked, the API my.getWifiList enables the mini-program to obtain SSID, BSSID, and signal strength information. Such information can be collectively leveraged to approximate user location (mapping observed access points to geolocation databases).

The privacy risks of accessing Wi-Fi scan data are well documented. Wi-Fi fingerprinting systems such as RADAR [28] have demonstrated that access point signatures can achieve location accuracy within a few meters. Subsequent studies have shown that Wi-Fi positioning works for both indoor and outdoor localization [29, 37, 50], and such data constitutes

```
1  public class H5WifiManagerPlugin extends H5SimplePlugin {
2    static final String GET_WIFI_LIST = "getWifiList";
3
4    private boolean getWifiList() {
5      List<ScanResult> results = WifiManager.getScanResults();
6      JSONArray list = new JSONArray();
7      for (ScanResult r : results) {
8        JSONObject obj = new JSONObject();
9        obj.put("SSID", r.SSID);
10       obj.put("BSSID", r.BSSID);
11       obj.put("signalStrength",
           ↪ WifiManager.calculateSignalLevel(r.level, 100));
12       obj.put("secure", r.capabilities.contains("WPA") ||
           ↪ r.capabilities.contains("WEP"));
13       list.add(obj);
14     }
15     JSONObject result = new JSONObject();
16     result.put("wifiList", list);
17     JSONObject data = new JSONObject();
18     data.put("data", result);
19     bridgeContext.sendBridgeResult("getWifiList", data, null);
20     return true;
21   }
22 }
```

Figure 5: A simplified implementation of `my.getWifiList` in Alipay.

Scope Category	Scope Enforcement							
	WeChat APIs		Baidu APIs		Alipay APIs		QQ APIs	
Get Device Info	wx.getNetworkType	✗	swan.getNetworkType	✗	my.getNetworkType	✓	qq.getNetworkType	✗
Location	wx.getWifiList	✓	swan.getWifiList	✓	my.getWifiList	✗		
	wx.connectWifi	✓	swan.connectWifi	✗	my.connectWifi	✗		
	wx.getConnectedWifi	✓	swan.getConnectedWifi	✗	my.getConnectedWifi	✗		
Camera	wx.chooseImage	✗	swan.chooseImage	✗	my.chooseImage	✓	qq.chooseImage	✓
	wx.scanCode	✗	swan.scanCode	✗	my.scan	✓	qq.scanCode	✗
	wx.chooseVideo	✗	swan.chooseVideo	✗			qq.chooseVideo	✗
	wx.chooseMedia	✗	swan.ai.faceDetect	✗				
			swan.ai.faceMatch	✗				
			swan.ai.faceSearch	✗				
Add Phone Contact	wx.addPhoneContact	✓	swan.addPhoneContact	✗	my.addPhoneContact	✗		
Invoice	wx.chooseInvoice	✗	swan.chooseInvoiceTitle	✓				
	wx.chooseInvoiceTitle	✗						
Address Info	wx.chooseAddress	✗	swan.chooseAddress	✓	my.getAddress	✓	qq.getAddress	✓

Table 10: Scope enforcement comparison across super-apps. ✗ means the scope enforcement is missing; ✓ means the scope enforcement is correctly implemented.

sensitive personal information that can be misused for surveillance and tracking.

Undeclared Biometric Authentication via startSoterAuthentication in WeChat. Figure 6 illustrates a vulnerability in WeChat that allows a malicious mini-program to invoke biometric authentication (e.g., fingerprint recognition) without declaring the required scope or obtaining user consent. The mini-program API `wx.startSoterAuthentication` enables access to the system's fingerprint sensor. Internally, the mini-program API accesses Android APIs in `FingerprintManager`, which requires the `USE_FINGERPRINT` permission.

However, our analysis shows that WeChat does not define any mini-program scope to regulate access to this functionality. Once the host app itself is granted `USE_FINGERPRINT` permission, mini-programs implicitly inherit this permission. As a result, a malicious mini-program can silently trigger fingerprint prompts, which may be misused for phishing, without requiring any prior disclosure or user opt-in. Based on reverse-engineering the internal API flow, we observe that `wx.startSoterAuthentication` invokes a chain of internal components, including the authentication UI activity `SoterAuthenticationUIWC` and its associated fingerprint controller (obfuscated as class `m` in package `yv3`), which ultimately constructs and issues a call to `FingerprintManager.authenticate`. Zhang et al. [58] found that many Android apps misused `FingerprintManager.authenticate`, triggering fingerprint prompts without permission checks or during inappropriate lifecycle stages, thus demonstrating that fingerprint authentication, if not properly protected, can be abused.

```
1  public class SoterAuthenticationUIWC extends MMActivity {
2    public void onCreate(Bundle savedInstanceState) {
3      FingerprintController controller =
       ↪  ControllerFactory.create(this, config, callback,
       ↪  SoterAuthenticationUIWC.m);
4      controller.initialize(savedInstanceState);
5    }
6  }
7
8  public class FingerprintController extends BaseSoterController {
9    public void initialize(Bundle bundle) {
10     if (!hasFingerprintPermission()) {
11       requestPermissions();
12     } else {
13       this.authReady = true;
14       startAuthentication();
15         // ↑ triggers FingerprintManager.authenticate()
16     }
17   }
18 }
```

Figure 6: Invocation chain of `wx.startSoterAuthentication` in WeChat.

7 Discussion

Privacy Policies and User Consent. Privacy policies [1, 2, 8] govern user consent and legal compliance, but are only effective when API scopes are properly specified. Developers rely on the correctness of scopes enforced by APIs when writing policies and specifying scopes for their mini-programs. For example, if a mini-program invokes an API (e.g., `getWifiList`) that misses a location-related scope, the developer will naturally not specify that their mini-program requires location permission. Thus, privacy policies cannot mitigate the permission enforcement gaps identified in our study.

Limitation and Future Work. Our analysis is scoped to the Android ecosystem. While permission inheritance risks may exist on other platforms, the vulnerabilities we identify are rooted in Android's architectural primitives: the loose coupling of IPC and implicit permission delegations via Binder. iOS utilizes a distinct XPC mechanism and Entitlement sys-

tem that do not share the same inheritance logic or attack surface as Android's component-based model. We focus on Android to ensure depth in measuring these logic flaws. Our study focuses on four super-apps that share similar framework designs (e.g., separate renderer and logic engine). Other platforms such as Grab and LINE have different designs where the mini-programs run inside a single WebView instance. Extending our analysis to iOS and these platforms is left for future work.

Suggestions. Given variations in API permission management across super-apps, the ecosystem would benefit from a unified mechanism for permission checks. Each super-app should ensure permissions are individually reviewed, especially when an API requires multiple permissions. super-apps should also maintain accurate documentation of which APIs require which permissions. To mitigate the identified access control gaps in Mini-program APIs, we also suggest an automated permission synthesizer as an IDE extension (e.g., for VS Code or WeChat DevTools), which leverages our analysis to suggest appropriate permission scopes during development. Implementing such an extension is beyond this paper's scope since super-app frameworks are closed-source.

8 Related Work

Mini-Program API Analysis and Security. Related to our work are APIDiff [47] and APIScope [48], which both study security issues in Mini-program APIs. However, they address fundamentally different problem settings from our work. APIDiff focuses on identifying cross-platform inconsistencies in mini-program API behaviors and permissions, whereas APIScope studies the hidden use of *undocumented* mini-program APIs and their associated scope controls.

Apinat [39] and ScopeChecker [31] are the closest prior work to PERMSCOPE, as all three systems address a similar research objective: the tools all aim to identify Mini-program APIs whose scopes do not properly reflect the underlying Android permissions. Nevertheless, the approaches differ substantially in several key aspects. (1) Coverage: Apinat and ScopeChecker are limited to SDK Android APIs. In contrast, PERMSCOPE performs static analysis of the Android framework (e.g., via ServiceManager registry and reachability analysis) to uncover both SDK and undocumented APIs, and other permission-protected resources (Intent-based interactions). Furthermore, the two approaches focus on a restricted subset of permissions (e.g., location), while PERMSCOPE analyzes all *normal* and *dangerous* permissions, defined in AOSP. Consequently, ScopeChecker's reported findings (38 instances) are concentrated in categories such as Camera, Album, Clipboard, and Location, while PERMSCOPE identifies inconsistencies spanning substantially broader resource categories. (2) Cross-process tracing: unlike ScopeChecker and Apinat, PERMSCOPE uniquely supports cross-process tracing by hooking Binder IPC, thereby enabling better coverage of reachable Android resources across process boundaries. (3) Test generation: PERMSCOPE uses LLM-based test generation to construct semantically valid mini-program API invocations and dynamically observe reachable Android resources. In contrast, Apinat relies on fuzzing-based input generation, leading to lower coverage due to input validity challenges. ScopeChecker, on the other hand, also uses LLM-based test generation; however for a different purpose: collecting scopes associated with super-app APIs, not to recover reachable Android APIs. It relies on Apinat for the latter task. (4) Permission map extraction: Apinat and ScopeChecker derive API permission mappings primarily from SDK documentation, whereas PERMSCOPE extracts the mappings statically from the framework. Overall, the three systems identify different issues in Mini-Program APIs, and are therefore complementary rather than redundant approaches.

Mini-Program Security. Prior work has studied mini-program security across ecosystem measurement, credential leakage, permission misuse, malware, and attack surfaces. Zhang et al. developed MiniCrawler [59], an open-source WeChat mini-program crawler that indexed over 1.33 million mini-programs. App Secret leak studies [60] showed that master key exposure can cause severe breaches. Other studies examined privileged API misuse under *identity confusion* [57], abusive permission request behaviors [51], cross-page request forgery vulnerabilities [61], and insecure resource management in mini-program cloud services [43]. Recent work further characterized mini-program malware [56], summarized emerging mini-program security challenges [55], detected malicious mini-programs abusing deceptive reporting interfaces [54], and exposed attacks against super-apps from desktop environments [49]. Unlike these studies, our work focuses on the systematic design and enforcement of mini-program API permissions.

Permission Mapping. Four prominent works [24, 26, 27, 33] have contributed to mapping Android APIs to permissions. *PScout* [26] and *Axplorer* [27] statically modeled and analyzed the Android middleware system services to generate a mapping of permissions to system service entry points (i.e., public interface APIs). *Arcade* [24] employed path-sensitive analysis and graph abstraction, while considering security checks such as caller UID and PID to improve the accuracy and coverage of the generated mappings. *Dynamo* [33] used dynamic testing for Java and native system APIs, accounting for path sensitivity, security check sequencing, and contextual information, hence leading to more accurate maps. Beyond recovering existing checks, *Poirot* [34] probabilistically recommends the access-control protections that under-protected Android framework entry points should enforce, and Vyas et al. [46] audit Android private framework APIs by inferring app-side security specifications, modeling the task as a probabilistic inference problem to flag APIs with inconsis-

tent access-control enforcement. More recently, Vagavolu et al. [45] apply learning over static execution paths to infer access-control recommendations

Dynamic Analysis on Android. Dynamic analysis is critical for identifying Android malware. Static analysis examines app permissions and features [40, 41], while *Droidapiminer* [22] extracts API-level features for malware detection. *DroidHook* [32] uses API-hooking to identify malware based on runtime activities. *DATDroid* [44] achieves high accuracy and low false positive rates. *DroidRA* [38] and *Amandroid* [52] focus on reflection and inter-component data flow analysis, respectively. *EntropLyzer* [36] uses entropy analysis of dynamic characteristics for malware classification.

9 Conclusion

In this paper, we identified four primary issues in the management of mini-program APIs and mini-program permissions across super-apps. We analyzed 2,067 APIs and found that WeChat, Baidu, Alipay, and QQ have 96, 33, 43, and 11 mini-program APIs, respectively, with missing scope checks. Across these four platforms, 183 APIs (8.85%) exhibit discrepancies in permission authorization. Our findings highlight the need for enhanced permission checks at the application interface layer within the super-app ecosystem and the absence of a unified privacy standard.

10 Acknowledgments

We thank all the reviewers for their valuable comments and guidance. We are also grateful to our shepherd, whose careful and constructive feedback throughout the revision process substantially improved this paper. The authors from The Ohio State University were partially supported by NSF Award 2330264. The authors from the University of Waterloo were supported in part by NSERC under grant RGPIN-0701 and by the Canada Foundation for Innovation under project 40236. This work benefited from the use of the CrySP RIPPLE Facility at the University of Waterloo.

Ethical Considerations

Our study potentially impacts super-app vendors and their users.

Stakeholders. This work affects several groups beyond the research team. End users of the four super-apps are the most directly affected; these platforms together serve more than a billion monthly active users, and many of them have no technical means to inspect what permissions a mini-program is using on their behalf. Super-app vendors are the second group, since they own the scope layer and are in a position to fix the gaps we identify. Mini-program developers are a third group: their privacy policies are written based on the scopes a super-app declares for each API, so a missing scope leads to incorrect policy declarations in their downstream mini-programs.

Research conduct. To follow the principle of beneficence, we restricted our experiments to a controlled environment. We used our own testing accounts, our own devices, and mini-programs we built ourselves. We interacted with the production APIs of the super-app platforms only to verify permission enforcement, and we rate-limited our requests to avoid any performance impact or service disruption. We accessed no real user data, no third-party mini-programs, and no information belonging to others; all permission gaps were verified against our own test identities only.

Disclosure and vendor response. We disclosed all identified vulnerabilities to the Security Response Centers of Tencent, Baidu, and Alipay before submission. All three vendors acknowledged our reports as security vulnerabilities and have fixed key issues. Alipay classified the issues as Medium severity and awarded a bounty for the disclosure.

Residual risk. All three vendors acknowledged our reports and have fixed key issues, but we do not have per-API patch-status confirmation from the vendors.

Open Science

We have made the source code of PERMSCOPE publicly available on GitHub at https://github.com/capy-zhiao/PERMSCOPE_, with a permanently archived version on Zenodo (https://doi.org/10.5281/zenodo.20498351).

References

[1] Alipay mini-program policy. https://opendocs.alipay.com/mini/03lwro.

[2] Baidu mini-program policy. https://smartprogram.baidu.com/docs/introduction/privacy_policy/.

[3] Binder. https://developer.android.com/reference/android/os/Binder.

[4] Install-time permissions on android. https://developer.android.com/guide/topics/permissions/overview#install-time.

[5] Miniapp standardization white paper version 2. https://www.w3.org/TR/mini-app-white-paper/. Accessed: 2024-04-29.

[6] my.getwifilist of alipay. https://opendocs.alipay.com/mini/api/getwifilist.

[7] Open capabilities / user information / authorization. https://developers.weixin.qq.com/miniprogram/en/dev/framework/open-ability/authorize.html.

[8] Wechat mini-program policy. https://developers.weixin.qq.com/miniprogram/en/dev/framework/user-privacy/miniprogram-intro.html.

[9] WALA: T. J. Watson Libraries for Analysis. [Online]. Available: https://wala.github.io/, 2023.

[10] Alipay Authorization Setting(Chinese version). [Online]. Available: https://opendocs.alipay.com/mini/api/qflu8f, 2024.

[11] Android permissions for system developers. [Online]. Available: https://android.googlesource.com/platform/frameworks/base/+/master/core/java/android/permission/Permissions.md, 2024.

[12] Baidu Authorization Setting(Chinese version). [Online]. Available: https://smartprogram.baidu.com/docs/develop/api/open/authorize_overview/, 2024.

[13] Intent class | android developers. https://cs.android.com/android/platform/superproject/+/android14-release:frameworks/base/core/java/android/content/Intent.java, 2024.

[14] iOS JavaScriptCore. [Online]. Available: https://developer.apple.com/documentation/javascriptcore, 2024.

[15] JEB Decompiler. [Online]. Available: https://www.pnfsoftware.com/jeb/, 2024.

[16] Manifest.permission | Android Developers. [Online]. Available: https://developer.android.com/reference/android/Manifest.permission, 2024.

[17] Open Capabilities / User Information / Authorization | WeChat Developers. [Online]. Available: https://developers.weixin.qq.com/miniprogram/dev/framework/open-ability/authorize.html, 2024.

[18] QQ Authorization Setting(Chinese version). [Online]. Available: https://q.qq.com/wiki/develop/miniprogram/frame/open_ability/open_userinfo.html#%E6%8E%88%E6%9D%83, 2024.

[19] WebView. [Online]. Available: https://developer.android.com/reference/android/webkit/WebView, 2024.

[20] WeChat scope. [Online]. Available: https://developers.weixin.qq.com/miniprogram/en/dev/framework/open-ability/authorize.html, 2024.

[21] Xposed Framework API. [Online]. Available: https://api.xposed.info/reference/packages.html, 2024.

[22] Yousra Aafer, Wenliang Du, and Heng Yin. Droidapiminer: Mining api-level features for robust malware detection in android. In *Security and Privacy in Communication Networks: 9th International ICST Conference, SecureComm 2013, Sydney, NSW, Australia, September 25-28, 2013, Revised Selected Papers 9*, pages 86–103. Springer, 2013.

[23] Yousra Aafer, Jianjun Huang, Yi Sun, Xiangyu Zhang, Ninghui Li, and Chen Tian. Acedroid: Normalizing diverse android access control checks for inconsistency detection. In *NDSS*, 2018.

[24] Yousra Aafer, Guanhong Tao, Jianjun Huang, Xiangyu Zhang, and Ninghui Li. Precise android api protection mapping derivation and reasoning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1151–1164, 2018.

[25] Yasemin Acar, Michael Backes, Sven Bugiel, Sascha Fahl, Patrick McDaniel, and Matthew Smith. Sok: Lessons learned from android security research for appified software platforms. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 433–451. IEEE, 2016.

[26] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. Pscout: analyzing the android permission

specification. In *Proceedings of the 2012 ACM conference on Computer and communications security*, pages 217–228, 2012.

[27] Michael Backes, Sven Bugiel, Erik Derr, Patrick McDaniel, Damien Octeau, and Sebastian Weisgerber. On demystifying the android application framework: Re-Visiting android permission specification analysis. In *25th USENIX security symposium (USENIX security 16)*, pages 1101–1118, 2016.

[28] Paramvir Bahl and Venkata N Padmanabhan. Radar: An in-building rf-based user location and tracking system. In *Proceedings IEEE INFOCOM 2000. Conference on computer communications. Nineteenth annual joint conference of the IEEE computer and communications societies (Cat. No. 00CH37064)*, volume 2, pages 775–784. Ieee, 2000.

[29] Chaimaa Basri and Ahmed El Khadimi. Survey on indoor localization system and recent advances of wifi fingerprinting technique. In *2016 5th international conference on multimedia computing and systems (ICMCS)*, pages 253–259. IEEE, 2016.

[30] Antonio Bianchi, Yanick Fratantonio, Aravind Machiry, Christopher Kruegel, Giovanni Vigna, Simon Pak Ho Chung, and Wenke Lee. Broken fingers: On the usage of the fingerprint api in android. In *NDSS*, 2018.

[31] Jiarui Che, Chenkai Guo, Naipeng Dong, Jiaqi Pei, Lingling Fan, Xun Mi, Xueshuo Xie, Xiangyang Luo, Zheli Liu, and Renhong Cheng. Uncovering api-scope misalignment in the app-in-app ecosystem. *Proc. ACM Softw. Eng.*, 2(ISSTA), June 2025.

[32] Yuning Cui, Yi Sun, and Zhaowen Lin. Droidhook: a novel api-hook based android malware dynamic analysis sandbox. *Automated Software Engineering*, 30(1):10, 2023.

[33] Abdallah Dawoud and Sven Bugiel. Bringing balance to the force: Dynamic analysis of the android application framework. 2021.

[34] Zeinab El-Rewini, Zhuo Zhang, and Yousra Aafer. Poirot: Probabilistically recommending protections for the android framework. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 937–950, 2022.

[35] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. Android permissions demystified. In *Proceedings of the 18th ACM conference on Computer and communications security*, pages 627–638, 2011.

[36] David Sean Keyes, Beiqi Li, Gurdip Kaur, Arash Habibi Lashkari, Francois Gagnon, and Frédéric Massicotte. Entroplyzer: Android malware classification and characterization using entropy analysis of dynamic characteristics. In *2021 Reconciling Data Analytics, Automation, Privacy, and Security: A Big Data Challenge (RDAAPS)*, pages 1–12. IEEE, 2021.

[37] Manikanta Kotaru, Kiran Joshi, Dinesh Bharadia, and Sachin Katti. Spotfi: Decimeter level localization using wifi. In *Proceedings of the 2015 ACM conference on special interest group on data communication*, pages 269–282, 2015.

[38] Li Li, Tegawendé F Bissyandé, Damien Octeau, and Jacques Klein. Droidra: Taming reflection to support whole-program analysis of android apps. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, pages 318–329, 2016.

[39] Haoran Lu, Luyi Xing, Yue Xiao, Yifan Zhang, Xiaojing Liao, XiaoFeng Wang, and Xueqiang Wang. Demystifying resource management risks in emerging mobile app-in-app ecosystems. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications Security*, pages 569–585, 2020.

[40] Hamida Lubuva, Qiming Huang, and Godfrey Charles Msonde. A review of static malware detection for android apps permission based on deep learning. *International Journal of Computer Networks and Applications*, 6(5):80–91, 2019.

[41] Samaneh Hosseini Moghaddam and Maghsood Abbaspour. Sensitivity analysis of static features for android malware detection. In *2014 22nd Iranian Conference on Electrical Engineering (ICEE)*, pages 920–924. IEEE, 2014.

[42] Joel Reardon, Álvaro Feal, Primal Wijesekera, Amit Elazari Bar On, Narseo Vallina-Rodriguez, and Serge Egelman. 50 ways to leak your data: An exploration of apps' circumvention of the android permissions system. In *28th USENIX security symposium (USENIX security 19)*, pages 603–620, 2019.

[43] Yizhe Shi, Zhemin Yang, Dingyi Liu, Kangwei Zhong, Jiarun Dai, and Min Yang. Better safe than sorry: Uncovering the insecure resource management in app-in-app cloud services. In *NDSS*, 2026.

[44] Rajan Thangaveloo, Wong Wan Jing, Chiew Kang Leng, and Johari Abdullah. Datdroid: Dynamic analysis technique in android malware detection. *Int. J. Adv. Sci. Eng. Inf. Technol*, 10(2):536–541, 2020.

[45] Dheeraj Vagavolu, Yousra Aafer, and Meiyappan Nagappan. (deep) learning of android access control recommendation from static execution paths. In *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, pages 758–772. IEEE, 2024.

[46] Parjanya Vyas, Asim Waheed, Yousra Aafer, and N Asokan. Auditing framework {APIs} via inferred app-side security specifications. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6061–6077, 2023.

[47] Chao Wang, Yue Zhang, and Zhiqiang Lin. One size does not fit all: Uncovering and exploiting cross platform discrepant APIs in WeChat. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6629–6646, 2023.

[48] Chao Wang, Yue Zhang, and Zhiqiang Lin. Uncovering and exploiting hidden apis in mobile super apps. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 2471–2485, 2023.

[49] Chao Wang, Yue Zhang, and Zhiqiang Lin. Rootfree attacks: Exploiting mobile platform's super apps from desktop. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pages 830–842, 2024.

[50] Jin Wang, Nicholas Tan, Jun Luo, and Sinno Jialin Pan. Woloc: Wifi-only outdoor localization using crowdsensed hotspot labels. In *IEEE INFOCOM 2017-IEEE Conference on Computer Communications*, pages 1–9. IEEE, 2017.

[51] Yin Wang, Ming Fan, Hao Zhou, Haijun Wang, Wuxia Jin, Jiajia Li, Wenbo Chen, Shijie Li, Yu Zhang, Deqiang Han, et al. Minichecker: Detecting data privacy risk of abusive permission request behavior in mini-programs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1667–1679, 2024.

[52] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. Amandroid: A precise and general inter-component data flow analysis framework for security vetting of android apps. *ACM Transactions on Privacy and Security (TOPS)*, 21(3):1–32, 2018.

[53] Fenghao Xu, Wenrui Diao, Zhou Li, Jiongyi Chen, and Kehuan Zhang. Badbluetooth: Breaking android security mechanisms via malicious bluetooth peripherals. *Proceedings 2019 Network and Distributed System Security Symposium*, 2019.

[54] Yuqing Yang and Zhiqiang Lin. Real or rogue? detecting malicious miniapps with deceptive reporting interface. In *Proceedings of the ACM Web Conference 2026*, pages 3159–3170, 2026.

[55] Yuqing Yang, Chao Wang, and Zhiqiang Lin. The rise of miniapps: A new frontier with security challenges in mobile apps. *IEEE Security & Privacy*, 2026.

[56] Yuqing Yang, Yue Zhang, and Zhiqiang Lin. Understanding miniapp malware: Identification, dissection, and characterization. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society, 2025.

[57] Lei Zhang, Zhibo Zhang, Ancong Liu, Yinzhi Cao, Xiaohan Zhang, Yanjun Chen, Yuan Zhang, Guangliang Yang, and Min Yang. Identity confusion in WebView-based mobile app-in-app ecosystems. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 1597–1613, Boston, MA, August 2022. USENIX Association.

[58] Xin Zhang, Xiaohan Zhang, Zhichen Liu, Bo Zhao, Zhemin Yang, and Min Yang. An empirical study on fingerprint api misuse with lifecycle analysis in real-world android apps. In *NDSS*, 2025.

[59] Yue Zhang, Bayan Turkistani, Allen Yuqing Yang, Chaoshun Zuo, and Zhiqiang Lin. A measurement study of wechat mini-apps. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 5(2):1–25, 2021.

[60] Yue Zhang, Yuqing Yang, and Zhiqiang Lin. Don't leak your keys: Understanding, measuring, and exploiting the appsecret leaks in mini-programs. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 2411–2425, 2023.

[61] Zidong Zhang, Qinsheng Hou, Lingyun Ying, Wenrui Diao, Yacong Gu, Rui Li, Shanqing Guo, and Haixin Duan. Minicat: Understanding and detecting cross-page request forgery vulnerabilities in mini-programs. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 525–539, 2024.